UNIVERSITY OF DELHI

ATMA RAM SANATAN DHARMA COLLEGE

*(Discipline Specific Core **2342012402**)*

Database Management System

PRACTICAL FILE

Submitted To:

MR. DHARMENDRA SINGH

Submitted By:

HARSH
24/48073
B.Sc. (H) Computer Science
ExaminationRollno:24003570045

Q. Create and use the following student-society database schema for a college using the standalone SQL editor to answer the given (sample) queries through a high-level language using ODBC connection.

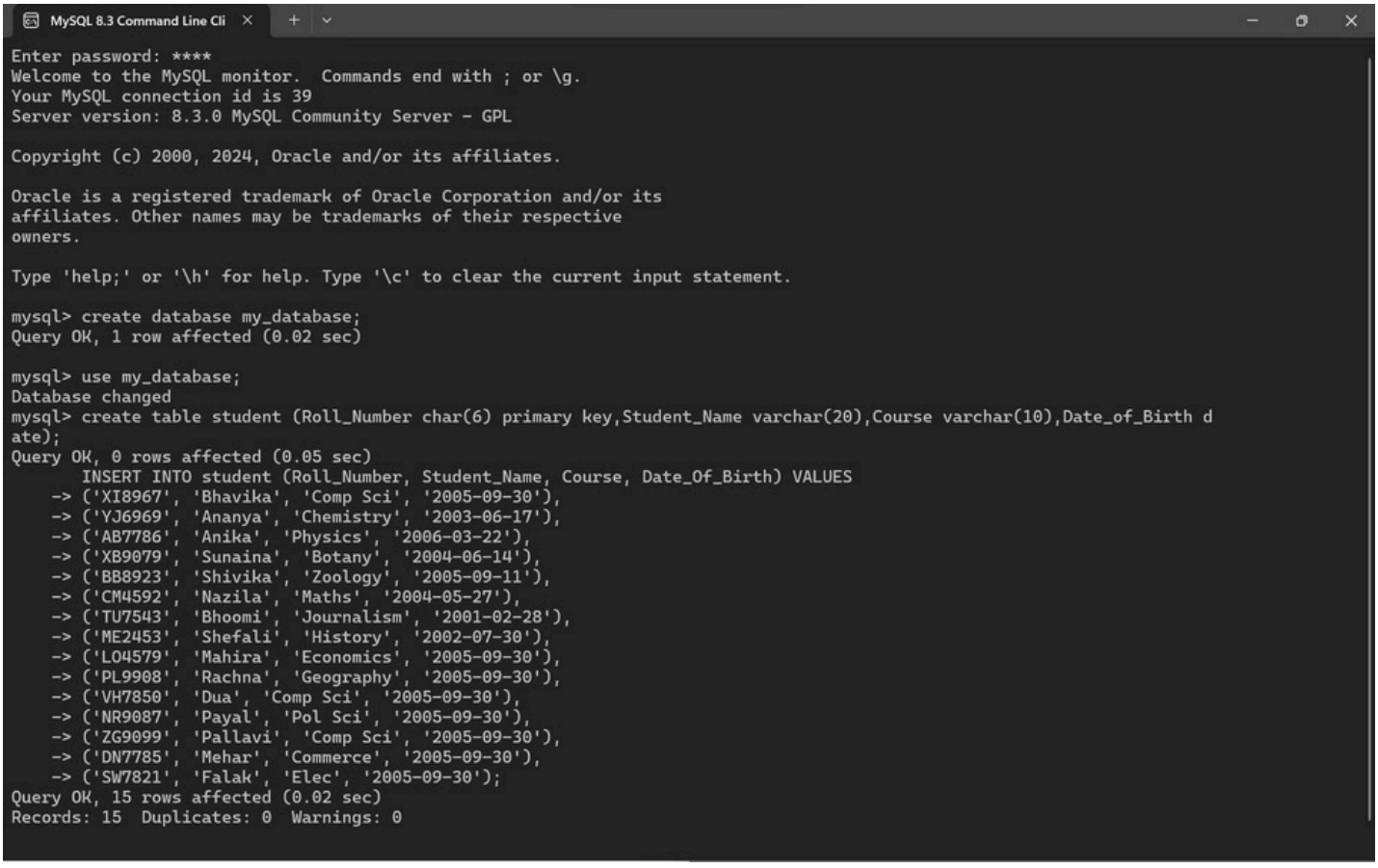
STUDENT	Roll No	StudentName	Course	DOB
	Char(6)	Varchar(20)	Varchar(10)	Date

SOCIETY	SocID	SocName	MentorName	TotalSeats
	Char(6)	Varchar(20)	Varchar(15)	Unsigned int

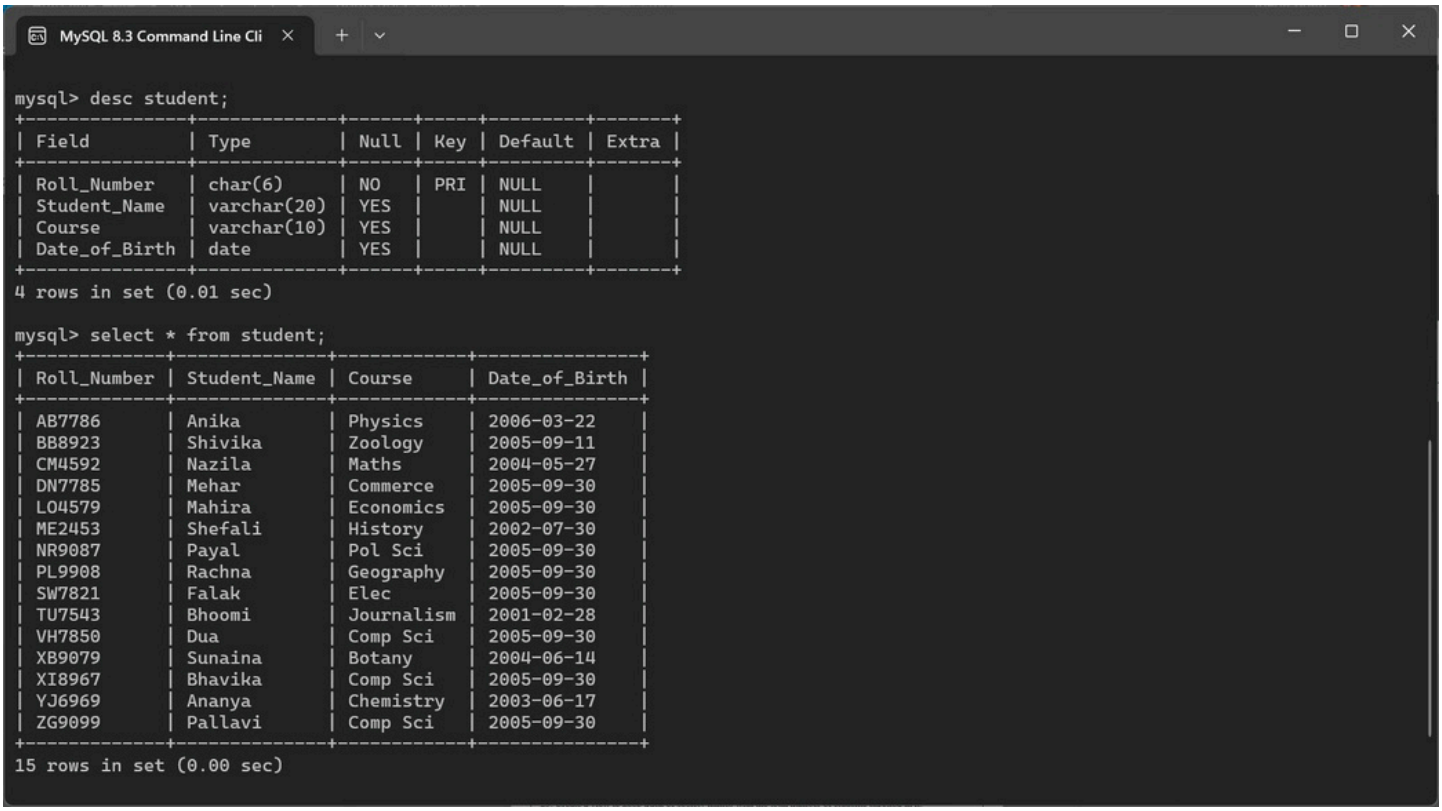
ENROLLMENT	Roll No	SID	DateOfEnrollment
	Char(6)	Char(6)	Date

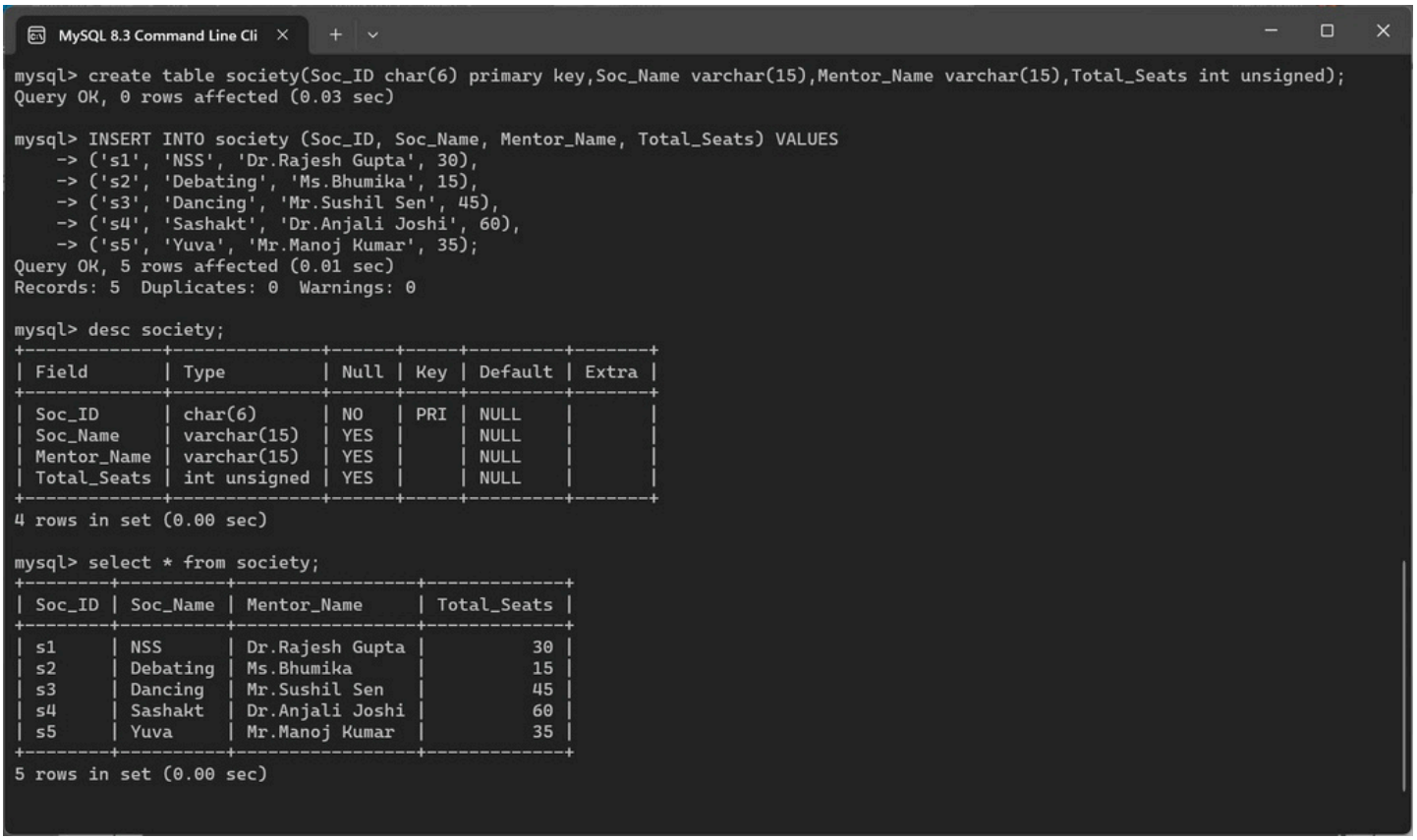
Here Rollno (ENROLLMENT) and SID (ENROLLMENT) are foreign keys.

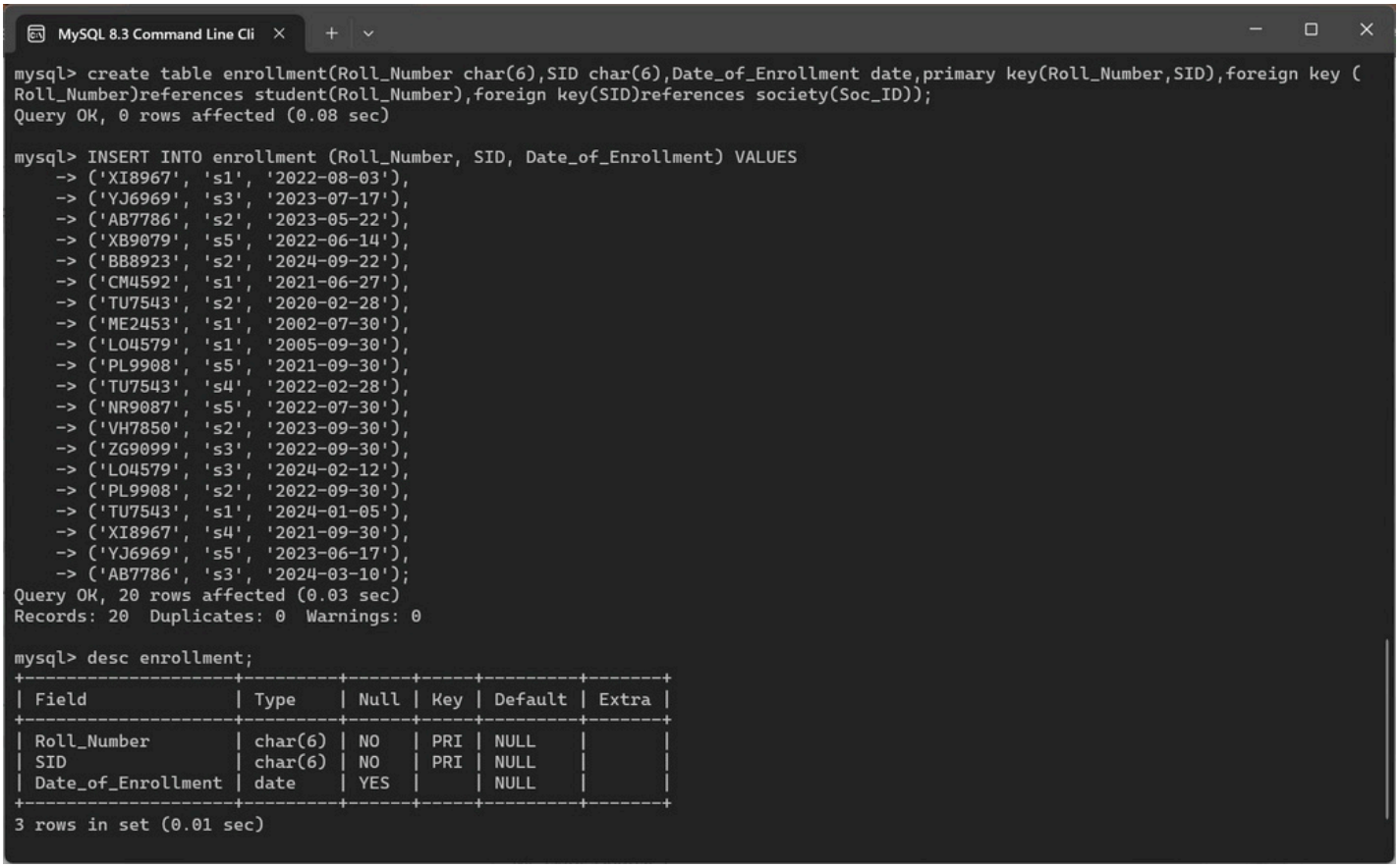
CREATING DATABASE AND TABLES

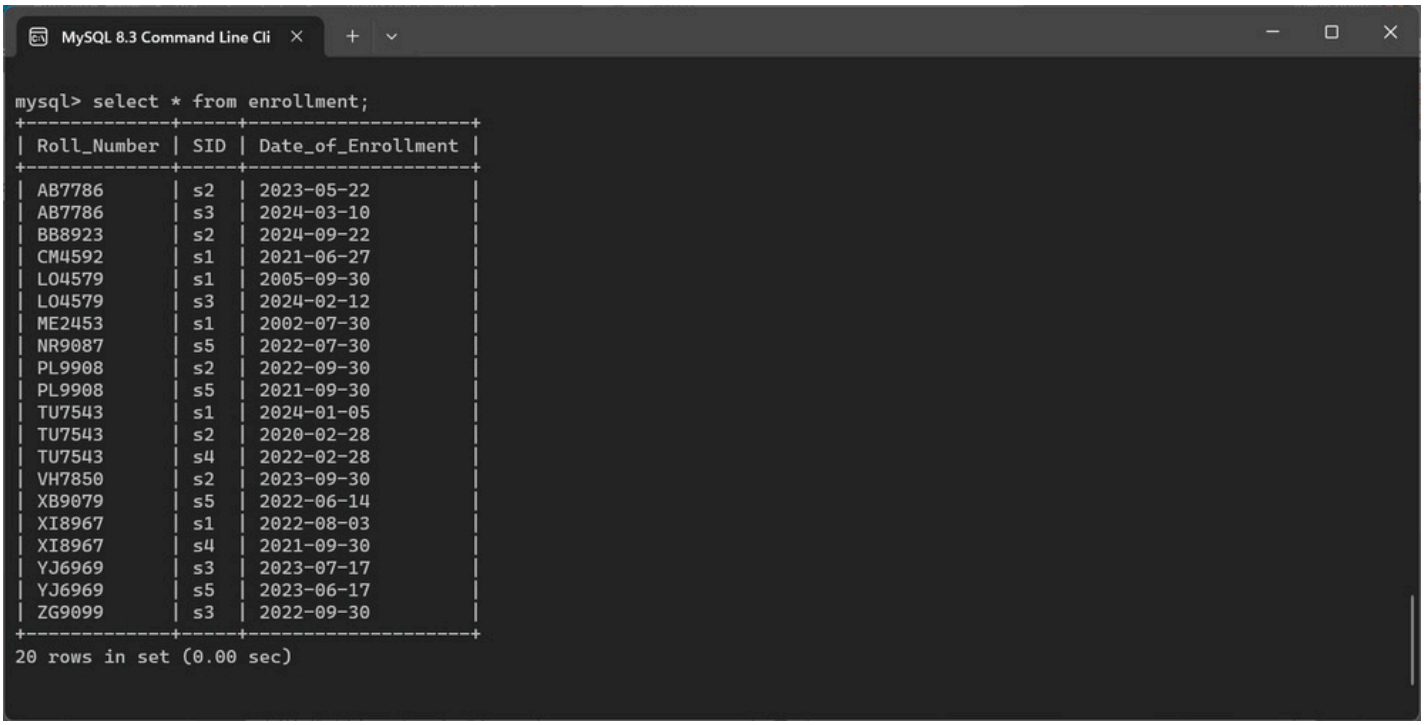


```
MySQL 8.3 Command Line Cli
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 39
Server version: 8.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database my_database;
Query OK, 1 row affected (0.02 sec)

mysql> use my_database;
Database changed
mysql> create table student (Roll_Number char(6) primary key,Student_Name varchar(20),Course varchar(10),Date_of_Birth d
ate);
Query OK, 0 rows affected (0.05 sec)
      INSERT INTO student (Roll_Number, Student_Name, Course, Date_Of_Birth) VALUES
    -> ('XI8967', 'Bhavika', 'Comp Sci', '2005-09-30'),
    -> ('YJ6969', 'Ananya', 'Chemistry', '2003-06-17'),
    -> ('AB7786', 'Anika', 'Physics', '2006-03-22'),
    -> ('XB9079', 'Sunaina', 'Botany', '2004-06-14'),
    -> ('BB8923', 'Shivika', 'Zoology', '2005-09-11'),
    -> ('CM4592', 'Nazila', 'Maths', '2004-05-27'),
    -> ('TU7543', 'Bhoomi', 'Journalism', '2001-02-28'),
    -> ('ME2453', 'Shefali', 'History', '2002-07-30'),
    -> ('LO4579', 'Mahira', 'Economics', '2005-09-30'),
    -> ('PL9908', 'Rachna', 'Geography', '2005-09-30'),
    -> ('VH7850', 'Dua', 'Comp Sci', '2005-09-30'),
    -> ('NR9087', 'Payal', 'Pol Sci', '2005-09-30'),
    -> ('ZG9099', 'Pallavi', 'Comp Sci', '2005-09-30'),
    -> ('DN7785', 'Mehar', 'Commerce', '2005-09-30'),
    -> ('SW7821', 'Falak', 'Elec', '2005-09-30');
Query OK, 15 rows affected (0.02 sec)
Records: 15  Duplicates: 0  Warnings: 0
```


MySQL 8.3 Command Line Cli
mysql> desc student;
+---------------+-------------+------+-----+---------+-------+
| Field | Type | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
Roll_Number	char(6)	NO	PRI	NULL	
Student_Name	varchar(20)	YES		NULL	
Course	varchar(10)	YES		NULL	
Date_of_Birth	date	YES		NULL	
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
mysql> select * from student;
+-------------+--------------+------------+---------------+
| Roll_Number | Student_Name | Course | Date_of_Birth |
+-------------+--------------+------------+---------------+
AB7786	Anika	Physics	2006-03-22
BB8923	Shivika	Zoology	2005-09-11
CM4592	Nazila	Maths	2004-05-27
DN7785	Mehar	Commerce	2005-09-30
LO4579	Mahira	Economics	2005-09-30
ME2453	Shefali	History	2002-07-30
NR9087	Payal	Pol Sci	2005-09-30
PL9908	Rachna	Geography	2005-09-30
SW7821	Falak	Elec	2005-09-30
TU7543	Bhoomi	Journalism	2001-02-28
VH7850	Dua	Comp Sci	2005-09-30
XB9079	Sunaina	Botany	2004-06-14
XI8967	Bhavika	Comp Sci	2005-09-30
YJ6969	Ananya	Chemistry	2003-06-17
ZG9099	Pallavi	Comp Sci	2005-09-30
+-------------+--------------+------------+---------------+
15 rows in set (0.00 sec)


MySQL 8.3 Command Line Cli
mysql> create table society(Soc_ID char(6) primary key,Soc_Name varchar(15),Mentor_Name varchar(15),Total_Seats int unsigned);
Query OK, 0 rows affected (0.03 sec)
mysql> INSERT INTO society (Soc_ID, Soc_Name, Mentor_Name, Total_Seats) VALUES
-> ('s1', 'NSS', 'Dr.Rajesh Gupta', 30),
-> ('s2', 'Debating', 'Ms.Bhumika', 15),
-> ('s3', 'Dancing', 'Mr.Sushil Sen', 45),
-> ('s4', 'Sashakt', 'Dr.Anjali Joshi', 60),
-> ('s5', 'Yuva', 'Mr.Manoj Kumar', 35);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
mysql> desc society;
+-------------+--------------+------+-----+---------+-------+
| Field | Type | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
Soc_ID	char(6)	NO	PRI	NULL	
Soc_Name	varchar(15)	YES		NULL	
Mentor_Name	varchar(15)	YES		NULL	
Total_Seats	int unsigned	YES		NULL	
+-------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
mysql> select * from society;
+--------+----------+-----------------+-------------+
| Soc_ID | Soc_Name | Mentor_Name | Total_Seats |
+--------+----------+-----------------+-------------+
s1	NSS	Dr.Rajesh Gupta	30
s2	Debating	Ms.Bhumika	15
s3	Dancing	Mr.Sushil Sen	45
s4	Sashakt	Dr.Anjali Joshi	60
s5	Yuva	Mr.Manoj Kumar	35
+--------+----------+-----------------+-------------+
5 rows in set (0.00 sec)


MySQL 8.3 Command Line Cli
mysql> create table enrollment(Roll_Number char(6),SID char(6),Date_of_Enrollment date,primary key(Roll_Number,SID),foreign key (
Roll_Number)references student(Roll_Number),foreign key(SID)references society(Soc_ID));
Query OK, 0 rows affected (0.08 sec)
mysql> INSERT INTO enrollment (Roll_Number, SID, Date_of_Enrollment) VALUES
-> ('XI8967', 's1', '2022-08-03'),
-> ('YJ6969', 's3', '2023-07-17'),
-> ('AB7786', 's2', '2023-05-22'),
-> ('XB9079', 's5', '2022-06-14'),
-> ('BB8923', 's2', '2024-09-22'),
-> ('CM4592', 's1', '2021-06-27'),
-> ('TU7543', 's2', '2020-02-28'),
-> ('ME2453', 's1', '2002-07-30'),
-> ('LO4579', 's1', '2005-09-30'),
-> ('PL9908', 's5', '2021-09-30'),
-> ('TU7543', 's4', '2022-02-28'),
-> ('NR9087', 's5', '2022-07-30'),
-> ('VH7850', 's2', '2023-09-30'),
-> ('ZG9099', 's3', '2022-09-30'),
-> ('LO4579', 's3', '2024-02-12'),
-> ('PL9908', 's2', '2022-09-30'),
-> ('TU7543', 's1', '2024-01-05'),
-> ('XI8967', 's4', '2021-09-30'),
-> ('YJ6969', 's5', '2023-06-17'),
-> ('AB7786', 's3', '2024-03-10');
Query OK, 20 rows affected (0.03 sec)
Records: 20 Duplicates: 0 Warnings: 0
mysql> desc enrollment;
Field	Type	Null	Key	Default	Extra
Roll_Number	char(6)	NO	PRI	NULL	
SID	char(6)	NO	PRI	NULL	
Date_of_Enrollment	date	YES		NULL	
3 rows in set (0.01 sec)


MySQL 8.3 Command Line Cli
mysql> select * from enrollment;
Roll_Number	SID	Date_of_Enrollment
AB7786	s2	2023-05-22
AB7786	s3	2024-03-10
BB8923	s2	2024-09-22
CM4592	s1	2021-06-27
LO4579	s1	2005-09-30
LO4579	s3	2024-02-12
ME2453	s1	2002-07-30
NR9087	s5	2022-07-30
PL9908	s2	2022-09-30
PL9908	s5	2021-09-30
TU7543	s1	2024-01-05
TU7543	s2	2020-02-28
TU7543	s4	2022-02-28
VH7850	s2	2023-09-30
XB9079	s5	2022-06-14
XI8967	s1	2022-08-03
XI8967	s4	2021-09-30
YJ6969	s3	2023-07-17
YJ6969	s5	2023-06-17
ZG9099	s3	2022-09-30
20 rows in set (0.00 sec)

Queries

1. Retrieve names of students enrolled in any society.
2. Retrieve all society names.
3. Retrieve students' names starting with letter 'A'.
4. Retrieve students' details studying in courses 'computer science' or 'chemistry'.
5. Retrieve students' names whose roll no either starts with 'X' or 'Z' and ends with '9'
6. Find society details with more than N TotalSeats where N is to be input by the user
7. Update society table for mentor name of a specific society
8. Find society names in which more than four students have enrolled
9. Find the name of youngest student enrolled in society 'NSS'
10. Find the name of most popular society (on the basis of enrolled students)
11. Find the name of two least popular societies (on the basis of enrolled students)
12. Find the student names who are not enrolled in any society
13. Find the student names enrolled in at least two societies
14. Find society names in which maximum students are enrolled
15. Find names of all students who have enrolled in any society and society names in which at least one student has enrolled
16. Find names of students who are enrolled in any of the three societies 'Debating', 'Dancing' and 'Sashakt'.
17. Find society names such that its mentor has a name with 'Gupta' in it.
18. Find the society names in which the number of enrolled students is only 10% of its capacity.
19. Display the vacant seats for each society.
20. Increment Total Seats of each society by 10%
21. Add the enrollment fees paid ('yes'/'No') field in the enrollment table.
22. Update date of enrollment of society id 's1' to '2018-01-15', 's2' to current date and 's3' to '2018-01-02'.
23. Create a view to keep track of society names with the total number of students enrolled in it.
24. Find student names enrolled in all the societies.
25. Count the number of societies with more than 5 students enrolled in it
26. Add column Mobile number in student table with default value '9999999999'
27. Find the total number of students whose age is > 20 years.
28. Find names of students who are born in 2001 and are enrolled in at least one society.
29. Count all societies whose name starts with 'S' and ends with 't' and at least 5 students are enrolled in the society.
30. Display the following information:

    Society name Mentor name Total Capacity Total Enrolled Unfilled Seats

PYTHON CODE :

```python
import mysql.connector as sq

conn = sq.connect(host='localhost', user='root', password='admin')
cur = conn.cursor()

# CREATING DATABASE AND TABLES
cur.execute("create database if not exists my_database")
cur.execute("use my_database")
cur.execute("create table if not exists student (Roll_Number char(6) primary key, Student_Name varchar(20), Course varchar(10), Date_of_Birth date)")
cur.execute("create table if not exists society(Soc_ID char(6) primary key, Soc_Name varchar(15), Mentor_Name varchar(15), Total_Seats int)")
cur.execute("create table if not exists enrollment(Roll_Number char(6), SID char(6), Date_of_Enrollment date, primary key(Roll_Number, SID)," \
"foreign key (Roll_Number) references student(Roll_Number), foreign key(SID) references society(Soc_ID))")

print("Tables Description")
print("\nTABLE: STUDENT\n", ('Field', 'Type', 'Null', 'Key', 'Default', 'Extra'), sep='')
cur.execute("desc student")
for i in cur:
    print(i)

print("\nTABLE: SOCIETY\n", ('Field', 'Type', 'Null', 'Key', 'Default', 'Extra'), sep='')
cur.execute("desc society")
for i in cur:
    print(i)

print("\nTABLE: ENROLLMENT\n", ('Field', 'Type', 'Null', 'Key', 'Default', 'Extra'), sep='')
cur.execute("desc enrollment")
for i in cur:
    print(i)

print("\n" + "-" * 60)

# INSERTING INTO TABLE STUDENT
cur.execute("insert into student values('XI8967', 'Bhavika', 'Comp Sci', '2005-09-30')")
cur.execute("insert into student values('YJ6969', 'Ananya', 'Chemistry', '2003-06-17')")
cur.execute("insert into student values('AB7786', 'Anika', 'Physics', '2006-03-22')")
cur.execute("insert into student values('XB9079', 'Sunaina', 'Botany', '2004-06-14')")
cur.execute("insert into student values('BB8923', 'Shivika', 'Zoology', '2005-09-11')")
cur.execute("insert into student values('CM4592', 'Nazila', 'Maths', '2004-05-27')")
cur.execute("insert into student values('TU7543', 'Bhoomi', 'Journalism', '2001-02-28')")
cur.execute("insert into student values('ME2453', 'Shefali', 'History', '2002-07-30')")
cur.execute("insert into student values('LO4579', 'Mahira', 'Economics', '2005-09-30')")
cur.execute("insert into student values('PL9908', 'Rachna', 'Geography', '2005-09-30')")
cur.execute("insert into student values('VH7850', 'Dua', 'Comp Sci', '2005-09-30')")
cur.execute("insert into student values('NR9087', 'Payal', 'Pol Sci', '2005-09-30')")
cur.execute("insert into student values('ZG9099', 'Pallavi', 'Comp Sci', '2005-09-30')")
cur.execute("insert into student values('DN7785', 'Mehar', 'Commerce', '2005-09-30')")
cur.execute("insert into student values('SW7821', 'Falak', 'Elec', '2005-09-30')")

print("Displaying Table Data")
print("\nTABLE: STUDENT\n", ('Roll_No', 'Stu_Name', 'Course', 'DOB'), sep='')
cur.execute("select * from student")
for i in cur:
    print(i)
```

```python
# INSERTING INTO TABLE SOCIETY
cur.execute("insert into society values('s1', 'NSS', 'Dr.Rajesh Gupta', '30')")
cur.execute("insert into society values('s2', 'Debating', 'Ms.Bhumika', '15')")
cur.execute("insert into society values('s3', 'Dancing', 'Mr.Sushil Sen', '45')")
cur.execute("insert into society values('s4', 'Sashakt', 'Dr.Anjali Joshi', '60')")
cur.execute("insert into society values('s5', 'Yuva', 'Mr.Manoj Kumar', '35')")

print("\nTABLE: Society\n", ('Soc_ID', 'Soc_Name', 'Mentor', 'TotalSeats'), sep='')
cur.execute("select * from society")
for i in cur:
    print(i)
```

```
    # INSERTING INTO TABLE ENROLLMENT
    cur.execute("insert into enrollment values('XI8967', 's1', '2022-08-03')")
    cur.execute("insert into enrollment values('YJ6969', 's3', '2023-07-17')")
    cur.execute("insert into enrollment values('AB7786', 's1', '2023-05-22')")
    cur.execute("insert into enrollment values('AB7786', 's2', '2021-05-22')")
    cur.execute("insert into enrollment values('XB9079', 's5', '2022-06-14')")
    cur.execute("insert into enrollment values('BB8923', 's2', '2024-09-22')")
    cur.execute("insert into enrollment values('CM4592', 's1', '2021-06-27')")
    cur.execute("insert into enrollment values('TU7543', 's2', '2020-02-28')")
    cur.execute("insert into enrollment values('ME2453', 's1', '2002-07-30')")
    cur.execute("insert into enrollment values('AB7786', 's4', '2023-05-22')")
    cur.execute("insert into enrollment values('LO4579', 's1', '2005-09-30')")
    cur.execute("insert into enrollment values('PL9908', 's5', '2021-09-30')")
    cur.execute("insert into enrollment values('TU7543', 's4', '2022-02-28')")
    cur.execute("insert into enrollment values('AB7786', 's5', '2022-06-27')")
    cur.execute("insert into enrollment values('NR9087', 's5', '2022-07-30')")
    cur.execute("insert into enrollment values('VH7850', 's2', '2023-09-30')")
    cur.execute("insert into enrollment values('ZG9099', 's3', '2022-09-30')")
    cur.execute("insert into enrollment values('LO4579', 's3', '2024-02-12')")
    cur.execute("insert into enrollment values('PL9908', 's2', '2022-09-30')")
    cur.execute("insert into enrollment values('TU7543', 's1', '2024-01-05')")
    cur.execute("insert into enrollment values('XI8967', 's4', '2021-09-30')")
    cur.execute("insert into enrollment values('YJ6969', 's5', '2023-06-17')")
    cur.execute("insert into enrollment values('AB7786', 's3', '2024-03-10')")

    print("\nTABLE: Enrollment\n", ('Roll_No', 'SID', 'Date_of_Enrollment'), sep='')
    cur.execute("select * from enrollment")
    for i in cur:
        print(i)

    print("\n" + "-" * 60)
    conn.commit()
```

```
    # Query 1
    print("\nQUERY1. Retrieve names of students enrolled in any society.")
    cur.execute("select distinct Student_Name from student, enrollment where student.Roll_Number = enrollment.Roll_Number")
    print("(Student Name)")
    for i in cur:
        print(i)

    # Query 2
    print("\nQUERY2. Retrieve all society names.")
    cur.execute("select Soc_Name from society;")
    print("(Society Name)")
    for i in cur:
        print(i)

    # Query 3
    print("\nQUERY3. Retrieve students' names starting with letter 'A'.")
    cur.execute("select Student_Name from student where Student_Name like 'A%';")
    print("(Student Name)")
    for i in cur:
        print(i)

    # Query 4
    print("\nQUERY4. Retrieve students details studying in courses 'computer science' or 'chemistry'.")
    cur.execute("select * from student where course in ('comp sci', 'chemistry')")
    print("(Roll Number, Student Name, Course, Date of Birth)")
    for i in cur:
        print(i)
```

```python

# Query 5
print("\nQUERY5. Retrieve students' names whose roll no either starts with 'X' or 'Z' and ends with '9'.")
cur.execute("select student_name from student where roll_number like 'x%9' or roll_number like 'z%9'")
print("(Student Name)")
for i in cur:
    print(i)

# Query 6
print("\nQUERY6. Find society details with more than N TotalSeats where N is to be input by the user.")
N = int(input("Enter total number of required seats: "))
cur.execute(f"select * from society where Total_Seats > {N}")
print("(Society ID, Society Name, Mentor Name, Total Seats)")
for i in cur:
    print(i)

# Query 7
print("\nQUERY7. Update society table for mentor name of a specific society.")
society_id = input("Enter Society ID: ")
new_mentor_name = input("Enter new mentor name: ")
cur.execute(f"update society set Mentor_Name = '{new_mentor_name}' where Soc_ID = '{society_id}'")
conn.commit()
cur.execute("Select * from society;")
print("(Society ID, Society Name, Mentor Name, Total Seats)")
for i in cur:
    print(i)

# Query 8
print("\nQUERY8. Find society names in which more than five students have enrolled.")
cur.execute("select soc_name from society where soc_id in (select sid from enrollment group by sid having count(*) > 4)")
print("(Society Name)")
for i in cur:
    print(i)

# Query 9
print("\nQUERY9. Find the name of youngest student enrolled in society 'NSS'.")
cur.execute("select student_name from student where roll_number in (select roll_number from enrollment where sid = 's1') order by date_of_birth desc limit 1")
print("(Youngest Student Name)")
for i in cur:
    print(i)

# Query 10
print("\nQUERY10. Find the name of most popular society (on the basis of enrolled students).")
cur.execute("select Soc_Name from society where Soc_ID = (select SID from enrollment group by SID order by count(*) desc limit 1)")
print("(Most Popular Society Name)")
for i in cur:
    print(i)

```

```python

# Query 11
print("\nQUERY11. Find the name of two least popular societies (on the basis of enrolled students).")
cur.execute("select s.soc_name from society s left join enrollment e on s.soc_id = e.sid group by s.soc_name order by count(e.roll_number) asc limit 2")
print("(Society Name)")
for i in cur:
    print(i)

# Query 12
print("\nQUERY12. Find the student names who are not enrolled in any society.")
cur.execute("select student_name from student where roll_number not in (select roll_number from enrollment)")
print("(Student Name)")
for i in cur:
    print(i)

# Query 13
print("\nQUERY13. Find the student names enrolled in at least two societies.")
cur.execute("select student_name from student where roll_number in (select roll_number from enrollment group by roll_number having count(*) >= 2)")
print("(Student Name)")
for i in cur:
    print(i)

# Query 14
print("\nQUERY14. Find society names in which maximum students are enrolled.")
cur.execute("select soc_name from society where soc_id = (select sid from enrollment group by sid order by count(*) desc limit 1);")
print("(Society Name)")
for i in cur:
    print(i)

# Query 15
print("\nQUERY15. Find names of all students who have enrolled in any society and society names in which at least one student has enrolled.")
cur.execute("select distinct s.student_name, so.soc_name from student s left join enrollment e on s.roll_number = e.roll_number left join society so on e.sid = so.soc_id;")
print("(Student Name, Society Name)")
for i in cur:
    print(i)

```

```
209
210 # Query 16
211 print("\nQUERY16. Find names of students who are enrolled in any of the three societies 'Debating', 'Dancing' and 'Sashakt'.")
212 cur.execute("select distinct Student_Name from student s join enrollment e on s.Roll_Number = e.Roll_Number join society so on e." \
213 "SID = so.Soc_ID where so.Soc_Name in ('Debating', 'Dancing', 'Sashakt')")
214 print("(Student Name)")
215 for i in cur:
216     print(i)
217
218 # Query 17
219 print("\nQUERY17. Find society names such that its mentor has a name with 'Gupta' in it.")
220 cur.execute("select soc_name from society where mentor_name like '%gupta%'")
221 print("(Society Name)")
222 for i in cur:
223     print(i)
224
225 # Query 18
226 print("\nQUERY18. Find the society names in which the number of enrolled students is only 10% of its capacity.")
227 cur.execute("select soc_name from society where total_seats * 0.1 > (select count(*) from enrollment where sid = society.soc_id)")
228 print("(Society Name)")
229 for i in cur:
230     print(i)
231
232 # Query 19
233 print("\nQUERY19. Display the vacant seats for each society.")
234 cur.execute("select so.soc_name, so.total_seats - count(e.roll_number) as vacant_seats from society so left join enrollment e on " \
235 "so.soc_id = e.sid group by so.soc_name, so.total_seats;")
236 print("(Society Name, Vacant Seats)")
237 for i in cur:
238     print(i)
239
240 # Query 20
241 print("\nQUERY20. Increment Total Seats of each society by 10%.")
242 cur.execute("update society set Total_Seats = Total_Seats * 1.1;")
243 conn.commit()
244 cur.execute("Select * from society")
245 print("(Society ID, Society Name, Mentor Name, Total Seats)")
246 for i in cur:
247     print(i)
248
249 # Query 21
250 print("\nQUERY21. Add the enrollment fees paid ('yes'/'No') field in the enrollment table.")
251 cur.execute("alter table enrollment add column Enrollment_Fees_Paid enum('Yes', 'No');")
252 cur.execute("desc enrollment")
253 print("('Field', 'Type', 'Null', 'Key', 'Default', 'Extra')")
254 for i in cur:
255     print(i)
256
```

```
257 # Query 22
258 print("\nQUERY22. Update date of enrollment of society id 's1' to '2018-01-15', 's2' to current date and 's3' to '2018-01-02'.")
259 cur.execute("update enrollment set Date_of_Enrollment = '2018-01-15' where SID = 's1'")
260 cur.execute("update enrollment set Date_of_Enrollment = now() where SID = 's2'")
261 cur.execute("update enrollment set Date_of_Enrollment = '2018-01-02' where SID = 's3'")
262 conn.commit()
263 cur.execute("select * from enrollment")
264 print("(Roll Number, SID, Date of Enrollment, Enrollment Fees Paid)")
265 for i in cur:
266     print(i)
267
268 # Query 23
269 print("\nQUERY23. Create a view to keep track of society names with the total number of students enrolled in it.")
270 cur.execute("create view Society_Enrollment as select so.Soc_Name, count(e.Roll_Number) as Total_Enrolled from society so left " \
271 "join enrollment e on so.Soc_ID = e.SID group by so.Soc_Name")
272 cur.execute("SELECT * FROM Society_Enrollment")
273 print("(Society Name, Total Enrolled)")
274 for i in cur:
275     print(i)
276
277 # Query 24
278 print("\nQUERY24. Find student names enrolled in all the societies.")
279 cur.execute("select Student_Name from student where Roll_Number in (select Roll_Number from enrollment group by Roll_Number having " \
280 "count(*) = (select count(*) from society))")
281 print("(Student Name)")
282 for i in cur:
283     print(i)
284
```

```
284
285 # Query 25
286 print("\nQUERY25. Count the number of societies with more than 5 students enrolled in it.")
287 cur.execute("select count(*) from (select SID from enrollment group by SID having count(*) > 5) as temp;")
288 print("(Count)")
289 for i in cur:
290     print(i)
291
292 # Query 26
293 print("\nQUERY26. Add column Mobile number in student table with default value '9999999999'.")
294 cur.execute("ALTER TABLE student ADD COLUMN Mobile_Number char(10) DEFAULT '9999999999'")
295 cur.execute("desc student")
296 print("(Field, Type, Null, Key, Default, Extra)")
297 for i in cur:
298     print(i)
299
300 # Query 27
301 print("\nQUERY27. Find the total number of students whose age is > 20 years.")
302 cur.execute("select count(*) from student where year(curdate()) - year(date_of_birth) > 20")
303 print("(Count)")
304 for i in cur:
305     print(i)
306
307 # Query 28
308 print("\nQUERY28. Find names of students who are born in 2001 and are enrolled in at least one society.")
309 cur.execute("select student_name from student where year(date_of_birth) = 2001 and roll_number in (select roll_number from enrollment);")
310 print("(Student Name)")
311 for i in cur:
312     print(i)
313
314 # Query 29
315 print("\nQUERY29. Count all societies whose name starts with 'S' and ends with 't' and at least 5 students are enrolled in the society.")
316 cur.execute("SELECT COUNT(*) FROM society WHERE Soc_Name LIKE 'S%t' AND Soc_ID IN (SELECT SID FROM enrollment GROUP BY SID HAVING COUNT(*) >= 5);")
317 print("(Count)")
318 for i in cur:
319     print(i)
320
321 # Query 30
322 print("\nQUERY30. Display the following information:\nSociety name\tMentor name\tTotal Capacity\tTotal Enrolled\tUnfilled Seats")
323 cur.execute("SELECT s.Soc_Name, s.Mentor_Name, s.Total_Seats, COUNT(e.Roll_Number) AS Total_Enrolled, (s.Total_Seats - COUNT(e.Roll_Number)) A" \
324 "S Unfilled_Seats FROM society s LEFT JOIN enrollment e ON s.Soc_ID = e.SID GROUP BY s.Soc_ID")
325 print("(Society Name, Mentor Name, Total Capacity, Total Enrolled, Unfilled Seats)")
326 for i in cur:
327     print(i)
328
```

OUTPUT:

```
Tables Description

TABLE: STUDENT
('Field', 'Type', 'Null', 'Key', 'Default', 'Extra')
('Roll_Number', 'char(6)', 'NO', 'PRI', None, '')
('Student_Name', 'varchar(20)', 'YES', '', None, '')
('Course', 'varchar(10)', 'YES', '', None, '')
('Date_of_Birth', 'date', 'YES', '', None, '')

TABLE: SOCIETY
('Field', 'Type', 'Null', 'Key', 'Default', 'Extra')
('Soc_ID', 'char(6)', 'NO', 'PRI', None, '')
('Soc_Name', 'varchar(15)', 'YES', '', None, '')
('Mentor_Name', 'varchar(15)', 'YES', '', None, '')
('Total_Seats', 'int', 'YES', '', None, '')

TABLE: ENROLLMENT
('Field', 'Type', 'Null', 'Key', 'Default', 'Extra')
('Roll_Number', 'char(6)', 'NO', 'PRI', None, '')
('SID', 'char(6)', 'NO', 'PRI', None, '')
('Date_of_Enrollment', 'date', 'YES', '', None, '')

------------------------------------------------------------
```

```
TABLE: STUDENT
('Roll_No', 'Stu_Name', 'Course', 'DOB')
('AB7786', 'Anika', 'Physics', datetime.date(2006, 3, 22))
('BB8923', 'Shivika', 'Zoology', datetime.date(2005, 9, 11))
('CM4592', 'Nazila', 'Maths', datetime.date(2004, 5, 27))
('DN7785', 'Mehar', 'Commerce', datetime.date(2005, 9, 30))
('LO4579', 'Mahira', 'Economics', datetime.date(2005, 9, 30))
('ME2453', 'Shefali', 'History', datetime.date(2002, 7, 30))
('NR9087', 'Payal', 'Pol Sci', datetime.date(2005, 9, 30))
('PL9908', 'Rachna', 'Geography', datetime.date(2005, 9, 30))
('SW7821', 'Falak', 'Elec', datetime.date(2005, 9, 30))
('TU7543', 'Bhoomi', 'Journalism', datetime.date(2001, 2, 28))
('VH7850', 'Dua', 'Comp Sci', datetime.date(2005, 9, 30))
('XB9079', 'Sunaina', 'Botany', datetime.date(2004, 6, 14))
('XI8967', 'Bhavika', 'Comp Sci', datetime.date(2005, 9, 30))
('YJ6969', 'Ananya', 'Chemistry', datetime.date(2003, 6, 17))
('ZG9099', 'Pallavi', 'Comp Sci', datetime.date(2005, 9, 30))

TABLE: Society
('Soc_ID', 'Soc_Name', 'Mentor', 'TotalSeats')
('s1', 'NSS', 'Dr.Rajesh Gupta', 30)
('s2', 'Debating', 'Ms.Bhumika', 15)
('s3', 'Dancing', 'Mr.Sushil Sen', 45)
('s4', 'Sashakt', 'Dr.Anjali Joshi', 60)
('s5', 'Yuva', 'Mr.Manoj Kumar', 35)

TABLE: Enrollment
('Roll_No', 'SID', 'Date_of_Enrollment')
('AB7786', 's1', datetime.date(2023, 5, 22))
('AB7786', 's2', datetime.date(2021, 5, 22))
('AB7786', 's3', datetime.date(2024, 3, 10))
('AB7786', 's4', datetime.date(2023, 5, 22))
('AB7786', 's5', datetime.date(2022, 6, 27))
('BB8923', 's2', datetime.date(2024, 9, 22))
('CM4592', 's1', datetime.date(2021, 6, 27))
('LO4579', 's1', datetime.date(2005, 9, 30))
('LO4579', 's3', datetime.date(2024, 2, 12))
('ME2453', 's1', datetime.date(2002, 7, 30))
('NR9087', 's5', datetime.date(2022, 7, 30))
('PL9908', 's2', datetime.date(2022, 9, 30))
('PL9908', 's5', datetime.date(2021, 9, 30))
('TU7543', 's1', datetime.date(2024, 1, 5))
('TU7543', 's2', datetime.date(2020, 2, 28))
('TU7543', 's4', datetime.date(2022, 2, 28))
('VH7850', 's2', datetime.date(2023, 9, 30))
('XB9079', 's5', datetime.date(2022, 6, 14))
('XI8967', 's1', datetime.date(2022, 8, 3))
('XI8967', 's4', datetime.date(2021, 9, 30))
('YJ6969', 's3', datetime.date(2023, 7, 17))
('YJ6969', 's5', datetime.date(2023, 6, 17))
('ZG9099', 's3', datetime.date(2022, 9, 30))

------------------------------------------------------------
```

```
------------------------------------------------------------

QUERY1. Retrieve names of students enrolled in any society.
(Student Name)
('Anika',)
('Shivika',)
('Nazila',)
('Mahira',)
('Shefali',)
('Payal',)
('Rachna',)
('Bhoomi',)
('Dua',)
('Sunaina',)
('Bhavika',)
('Ananya',)
('Pallavi',)

QUERY2. Retrieve all society names.
(Society Name)
('NSS',)
('Debating',)
('Dancing',)
('Sashakt',)
('Yuva',)

QUERY3. Retrieve students' names starting with letter 'A'.
(Student Name)
('Anika',)
('Ananya',)

QUERY4. Retrieve students details studying in courses 'computer science' or 'chemistry'.
(Roll Number, Student Name, Course, Date of Birth)
('VH7850', 'Dua', 'Comp Sci', datetime.date(2005, 9, 30))
('XI8967', 'Bhavika', 'Comp Sci', datetime.date(2005, 9, 30))
('YJ6969', 'Ananya', 'Chemistry', datetime.date(2003, 6, 17))
('ZG9099', 'Pallavi', 'Comp Sci', datetime.date(2005, 9, 30))

QUERY5. Retrieve students' names whose roll no either starts with 'X' or 'Z' and ends with '9'.
(Student Name)
('Sunaina',)
('Pallavi',)
```

```
QUERY6. Find society details with more than N TotalSeats where N is to be input by the user.
Enter total number of required seats: 22
(Society ID, Society Name, Mentor Name, Total Seats)
('s1', 'NSS', 'Dr.Rajesh Gupta', 30)
('s3', 'Dancing', 'Mr.Sushil Sen', 45)
('s4', 'Sashakt', 'Dr.Anjali Joshi', 60)
('s5', 'Yuva', 'Mr.Manoj Kumar', 35)

QUERY7. Update society table for mentor name of a specific society.
Enter Society ID: s3
Enter new mentor name: Mr. Manas Das
(Society ID, Society Name, Mentor Name, Total Seats)
('s1', 'NSS', 'Dr.Rajesh Gupta', 30)
('s2', 'Debating', 'Ms.Bhumika', 15)
('s3', 'Dancing', 'Mr. Manas Das', 45)
('s4', 'Sashakt', 'Dr.Anjali Joshi', 60)
('s5', 'Yuva', 'Mr.Manoj Kumar', 35)

QUERY8. Find society names in which more than five students have enrolled.
(Society Name)
('NSS',)
('Debating',)
('Yuva',)
```

```
QUERY9. Find the name of youngest student enrolled in society 'NSS'.
(Youngest Student Name)
('Anika',)

QUERY10. Find the name of most popular society (on the basis of enrolled students).
(Most Popular Society Name)
('NSS',)

QUERY11. Find the name of two least popular societies (on the basis of enrolled students).
(Society Name)
('Sashakt',)
('Dancing',)

QUERY12. Find the student names who are not enrolled in any society.
(Student Name)
('Mehar',)
('Falak',)
```

```
QUERY13. Find the student names enrolled in at least two societies.
(Student Name)
('Anika',)
('Mahira',)
('Rachna',)
('Bhoomi',)
('Bhavika',)
('Ananya',)

QUERY14. Find society names in which maximum students are enrolled.
(Society Name)
('NSS',)

QUERY15. Find names of all students who have enrolled in any society and society names in which at least one student has enrolled.
(Student Name, Society Name)
('Anika', 'NSS')
('Anika', 'Debating')
('Anika', 'Dancing')
('Anika', 'Sashakt')
('Anika', 'Yuva')
('Shivika', 'Debating')
('Nazila', 'NSS')
('Mehar', None)
('Mahira', 'NSS')
('Mahira', 'Dancing')
('Shefali', 'NSS')
('Payal', 'Yuva')
('Rachna', 'Debating')
('Rachna', 'Yuva')
('Falak', None)
('Bhoomi', 'NSS')
('Bhoomi', 'Debating')
('Bhoomi', 'Sashakt')
('Dua', 'Debating')
('Sunaina', 'Yuva')
('Bhavika', 'NSS')
('Bhavika', 'Sashakt')
('Ananya', 'Dancing')
('Ananya', 'Yuva')
('Pallavi', 'Dancing')

QUERY16. Find names of students who are enrolled in any of the three societies 'Debating', 'Dancing' and 'Sashakt'.
(Student Name)
('Anika',)
('Shivika',)
('Rachna',)
('Bhoomi',)
('Dua',)
('Mahira',)
('Ananya',)
('Pallavi',)
('Bhavika',)
```

```
QUERY17. Find society names such that its mentor has a name with 'Gupta' in it.
(Society Name)
('NSS',)

QUERY18. Find the society names in which the number of enrolled students is only 10% of its capacity.
(Society Name)
('Dancing',)
('Sashakt',)

QUERY19. Display the vacant seats for each society.
(Society Name, Vacant Seats)
('NSS', 24)
('Debating', 10)
('Dancing', 41)
('Sashakt', 57)
('Yuva', 30)

QUERY20. Increment Total Seats of each society by 10%.
(Society ID, Society Name, Mentor Name, Total Seats)
('s1', 'NSS', 'Dr.Rajesh Gupta', 33)
('s2', 'Debating', 'Ms.Bhumika', 17)
('s3', 'Dancing', 'Mr. Manas Das', 50)
('s4', 'Sashakt', 'Dr.Anjali Joshi', 66)
('s5', 'Yuva', 'Mr.Manoj Kumar', 39)

QUERY21. Add the enrollment fees paid ('yes'/'No') field in the enrollment table.
('Field', 'Type', 'Null', 'Key', 'Default', 'Extra')
('Roll_Number', 'char(6)', 'NO', 'PRI', None, '')
('SID', 'char(6)', 'NO', 'PRI', None, '')
('Date_of_Enrollment', 'date', 'YES', '', None, '')
('Enrollment_Fees_Paid', "enum('Yes','No')", 'YES', '', None, '')

QUERY22. Update date of enrollment of society id 's1' to '2018-01-15', 's2' to current date and 's3' to '2018-01-02'.
(Roll Number, SID, Date of Enrollment, Enrollment Fees Paid)
('AB7786', 's1', datetime.date(2018, 1, 15), None)
('AB7786', 's2', datetime.date(2026, 4, 22), None)
('AB7786', 's3', datetime.date(2018, 1, 2), None)
('AB7786', 's4', datetime.date(2023, 5, 22), None)
('AB7786', 's5', datetime.date(2022, 6, 27), None)
('BB8923', 's2', datetime.date(2026, 4, 22), None)
('CM4592', 's1', datetime.date(2018, 1, 15), None)
('LO4579', 's1', datetime.date(2018, 1, 15), None)
('LO4579', 's3', datetime.date(2018, 1, 2), None)
('ME2453', 's1', datetime.date(2018, 1, 15), None)
('NR9087', 's5', datetime.date(2022, 7, 30), None)
('PL9908', 's2', datetime.date(2026, 4, 22), None)
('PL9908', 's5', datetime.date(2021, 9, 30), None)
('TU7543', 's1', datetime.date(2018, 1, 15), None)
('TU7543', 's2', datetime.date(2026, 4, 22), None)
('TU7543', 's4', datetime.date(2022, 2, 28), None)
('VH7850', 's2', datetime.date(2026, 4, 22), None)
('XB9079', 's5', datetime.date(2022, 6, 14), None)
('XI8967', 's1', datetime.date(2018, 1, 15), None)
('XI8967', 's4', datetime.date(2021, 9, 30), None)
('YJ6969', 's3', datetime.date(2018, 1, 2), None)
('YJ6969', 's5', datetime.date(2023, 6, 17), None)
('ZG9099', 's3', datetime.date(2018, 1, 2), None)
```

```
QUERY23. Create a view to keep track of society names with the total number of students enrolled in it.
(Society Name, Total Enrolled)
('NSS', 6)
('Debating', 5)
('Dancing', 4)
('Sashakt', 3)
('Yuva', 5)

QUERY24. Find student names enrolled in all the societies.
(Student Name)
('Anika',)

QUERY25. Count the number of societies with more than 5 students enrolled in it.
(Count)
(1,)

QUERY26. Add column Mobile number in student table with default value '9999999999'.
(Field, Type, Null, Key, Default, Extra)
('Roll_Number', 'char(6)', 'NO', 'PRI', None, '')
('Student_Name', 'varchar(20)', 'YES', '', None, '')
('Course', 'varchar(10)', 'YES', '', None, '')
('Date_of_Birth', 'date', 'YES', '', None, '')
('Mobile_Number', 'char(10)', 'YES', '', '9999999999', '')

QUERY27. Find the total number of students whose age is > 20 years.
(Count)
(14,)

QUERY28. Find names of students who are born in 2001 and are enrolled in at least one society.
(Student Name)
('Bhoomi',)

QUERY29. Count all societies whose name starts with 'S' and ends with 't' and at least 5 students are enrolled in the society.
(Count)
(0,)

QUERY30. Display the following information:
Society name    Mentor name     Total Capacity  Total Enrolled  Unfilled Seats
(Society Name, Mentor Name, Total Capacity, Total Enrolled, Unfilled Seats)
('NSS', 'Dr.Rajesh Gupta', 33, 6, 27)
('Debating', 'Ms.Bhumika', 17, 5, 12)
('Dancing', 'Mr. Manas Das', 50, 4, 46)
('Sashakt', 'Dr.Anjali Joshi', 66, 3, 63)
('Yuva', 'Mr.Manoj Kumar', 39, 5, 34)
```